

PGDIT-108

WEB PROGRAMMING

ECMAScript 6

JS

Institute of Information Technology

Lecture 04

JavaScript

JAVASCRIPT

- JavaScript is a cross-platform, object-oriented scripting language used to make webpages interactive (e.g., having complex animations, clickable buttons, popup menus, etc.).
- JavaScript can be inserted into HTML pages and executed by all modern web browsers.

Why is JavaScript used/required?

- Validate client-side data
- Change color or styles of HTML elements
- Reacting to an event

JavaScript code added to HTML can perform a wide variety of functions:

- Decision making
- Submitting forms.
- Performing complex mathematical calculations.
- Data entry validation, ...etc.

ADVANTAGES & LIMITATIONS

1. Less server interaction: You can validate user input before sending the page off to the server. This saves server traffic, which means less load on your server.
2. Immediate feedback to the visitors: They don't have to wait for a page reload to see if they have forgotten to enter something.
3. Increased interactivity: You can create interfaces that react when the user hovers over them with a mouse or activates them via the keyboard.
4. Richer interfaces: You can use JavaScript to include such items as drag-and-drop components and sliders to give a Rich Interface to your site visitors.

1. Client-side JavaScript does not allow the reading or writing of files. This has been kept for security reasons.
2. Large, complex applications can become messy without frameworks/libraries (like React, Angular, Vue).
3. Slower compared to low-level languages (like C, C++).
4. JavaScript doesn't have any multithreading or multiprocessing capabilities.
5. Search engines sometimes struggle to crawl JavaScript-heavy pages.
6. If JavaScript is disabled in a browser, scripts won't run.

HOW TO INSERT JAVASCRIPT CODE IN HTML

- JavaScripts in HTML must be inserted between `<script>` and `</script>` tags.
- JavaScripts can be put in the `<body>` and in the `<head>` section of an HTML page or write in external page.

The `<script>` Tag

- To insert a JavaScript into an HTML page, use the `<script>` tag.
- The `<script>` and `</script>` tells where the JavaScript starts and ends.

The lines between the `<script>` and `</script>` contain the JavaScript:

```
<script>  
alert("My First JavaScript");  
</script>
```

A JAVASCRIPT FUNCTION IN <HEAD>

```
<!DOCTYPE html>
<html>
  <head>
    <script>
      function myFunction() {
        alert("Hello World");
      }
    </script>
  </head>
  <body>
    <h1>My Web Page</h1>
    <p id="demo">A Paragraph</p>
    <button type="button" onclick="myFunction()">Try it</button>
  </body>
</html>
```

A JAVASCRIPT FUNCTION IN <BODY>

```
<!DOCTYPE html>
<html>
  <head> </head>
  <body>
    <h1>My Web Page</h1>
    <p>
      Lorem ipsum dolor sit amet consectetur adipisicing elit.</p>
    <button type="button" onclick="myFunction()">Try it</button>
    <script>
      function myFunction() {
        alert("Hello World");
      }
    </script>
  </body>
</html>
```

EXTERNAL JAVASCRIPT

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<script src="script.js"> </script>
```

```
</head>
```

```
<body>
```

```
</body>
```

```
</html>
```

JavaScript in <head>



Pros:

1. Ensures the script is loaded before the page content.
2. Useful for **global functions** and **variables**.
3. Functions declared in <head> can be accessed anywhere in the page.



Cons:

- Browser stops loading HTML until the script finishes.
- Can cause **slow page loading**.
- Scripts may try to access HTML elements that **do not exist yet**.
- Can lead to **JavaScript errors**.
- Heavy scripts in <head> reduce performance.

JavaScript in <body>



Pros:

- The page **loads first**, then the script runs.
- Better for **performance and user experience**.



Cons:

- If the script is needed early (before content loads), this may cause errors.

Best Practice Today

👉 Place scripts **just before the closing `</body>` tag**
OR **use modern attributes:**

With `defer`:

```
<script src="script.js" defer></script>
```

- Loads script in the background.
- Executes after HTML is fully parsed.

With `async`:

```
<script src="script.js" async></script>
```

- Loads script in parallel.
- Executes **as soon as it's ready** (may not follow order).

Example

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>JavaScript Placement Demo</title>

    <!-- 1. Script in HEAD -->
    <script>
      console.log("❶ Script in <head> runs before page content loads.");
      alert("❶ Script in HEAD executed");
    </script>

    <!-- 2. Script in HEAD with defer -->
    <script defer>
      console.log("❷ Script in <head> with defer runs after HTML parsing.");
      alert("❷ Script in HEAD with defer executed");
    </script>

    <!-- 3. Script in HEAD with async -->
    <script async>
      console.log(
        "❸ Script in <head> with async runs independently (order not guaranteed).";
      );
      alert("❸ Script in HEAD with async executed");
    </script>
  </head>
```

```
<body>
  <h1>JavaScript Placement Test</h1>
  <p>
    Open the browser console (F12 → Console) and observe script
    execution
    order.
  </p>

  <!-- 4. Script inside BODY -->
  <script>
    console.log(
      "❹ Script in <body> runs after HTML is parsed up to this point."
    );
    alert("❹ Script in BODY executed");
  </script>

  <!-- 5. Script at end of BODY (Best Practice) -->
  <script>
    console.log("❺ Script at the END of <body> runs after full HTML
    loads.");
    alert("❺ Script at END of BODY executed");
  </script>
</body>
</html>
```

OUTPUTS

JavaScript can "display" data in different ways:

- ❑ Writing into an alert box, using **window.alert()**.
- ❑ Writing into the HTML output using **document.write()**.
- ❑ Writing into an HTML element, using **innerHTML**.
- ❑ Writing into the browser console, using **console.log()**.

KEYWORDS

Keyword	Description
break	Terminates a switch or a loop
continue	Jumps out of a loop and starts at the top
debugger	Stops the execution of JavaScript, and calls (if available) the debugging function
do ... while	Executes a block of statements, and repeats the block, while a condition is true
for	Marks a block of statements to be executed, as long as a condition is true
function	Declares a function
if ... else	Marks a block of statements to be executed, depending on a condition
return	Exits a function
switch	Marks a block of statements to be executed, depending on different cases
try ... catch	Implements error handling to a block of statements
var	Declares a variable

MATH OBJECT METHODS

<u>Method</u>	<u>Description</u>
abs(x)	Returns the absolute value of x
acos(x)	Returns the arccosine of x, in radians
asin(x)	Returns the arcsine of x, in radians
atan(x)	Returns the arctangent of x as a numeric value between $-\pi/2$ and $\pi/2$ radians
atan2(y,x)	Returns the arctangent of the quotient of its arguments
ceil(x)	Returns x, rounded upwards to the nearest integer
cos(x)	Returns the cosine of x (x is in radians)
exp(x)	Returns the value of E^x
floor(x)	Returns x, rounded downwards to the nearest integer
log(x)	Returns the natural logarithm (base E) of x
max(x,y,z,...,n)	Returns the number with the highest value
min(x,y,z,...,n)	Returns the number with the lowest value
pow(x,y)	Returns the value of x to the power of y
random()	Returns a random number between 0 and 1
round(x)	Rounds x to the nearest integer
sin(x)	Returns the sine of x (x is in radians)
sqrt(x)	Returns the square root of x
tan(x)	Returns the tangent of an angle

JAVASCRIPT VARIABLES

JavaScript Variables can be declared in 4 ways:

- Automatically
 - Using **var**
 - Using **let**
 - Using **const**
-
- The **var** keyword was used in all JavaScript code from 1995 to 2015.
 - The **let** and **const** keywords were added to JavaScript in 2015.
 - The **var** keyword should only be used in code written for older browsers.

VARIABLE AUTOMATICALLY

They are automatically declared when first used:

```
x = 5;
```

```
y = 6;
```

```
z = x + y;
```

Note

It is considered good programming practice to always declare variables before use.

JavaScript Let

- The let keyword was introduced in ES6 (2015)
- Variables defined with **let** cannot be Redeclared
- Variables defined with **let** must be Declared before use
- Variables defined with **let** have Block Scope
- Can be reassigned.

Cannot be Redeclared:

```
let count = 5;  
count = 10; // allowed
```

Variables defined with let can not be redeclared.

You can not accidentally redeclare a variable declared with **let**.

With **let** you can not do this:

```
let x = "John Doe";  
let x = 0;
```

With **var** you can:

```
var x = "John Doe";  
var x = 0;
```

Block Scope

ECMAScript 6

JS

Before ES6 (2015), JavaScript had **Global Scope** and **Function Scope**.

ES6 introduced two important new JavaScript keywords: **let** and **const**.

These **two keywords** provide **Block Scope** in JavaScript.

Variables declared with the **var** keyword **can NOT** have **block scope**.

Variables declared **let** inside a **{ }** block cannot be accessed from outside the block:

```
{  
  let x = 2;  
}
```

// x can NOT be used here

Variables declared **var** inside a **{ }** block can be accessed from outside the block.

```
{  
  var x = 2;  
}
```

// x CAN be used here

USING CONST & LET

```
const price1 = 5;  
const price2 = 6;  
let total = price1 + price2;
```

- The two variables **price1** and **price2** are declared with the **const** keyword.
- These are **constant values** and cannot be **changed**.
- The variable **total** is declared with the **let** keyword.
- The value **total** can be changed.

JAVASCRIPT VARIABLES & DATA TYPES

- JavaScript variables can be declare with “**var**” keyword
- Example:
 - `var carname=“Toyota”;`
- Variable can be string, number, float.
- To assign string into a variable use double quotation.
- To assign number into a variable do not use any quotation.
- Example:

```
var pi=3.14;           //float
var person="John Doe"; //string
var val=10;            //number
```

- JavaScript variables can hold many **data types**: numbers, strings, arrays, objects and more:
 - Example of dynamic types

```
var x;           // Now x is undefined
var x = 5;       // Now x is a Number
var x = "John";  // Now x is a String
```

When to Use var, let, or const?

- Always declare variables
- Always use const if the value should not be changed
- Always use const if the type should not be changed (Arrays and Objects)
- Only use let if you can't use const
- Only use var if you **MUST** support old browsers.

JAVASCRIPT ARRAYS

1st Way:

```
var cars=new Array();  
cars[0]="Saab";  
cars[1]="Volvo";  
cars[2]="BMW";
```

2nd Way:

```
var cars=new Array("Saab","Volvo","BMW");
```

3rd Way:

```
var cars=["Saab","Volvo","BMW"];
```

All three ways will work same and create JavaScript array.

ARRAY METHODS

- **valueOf()**-- returns an array as a string.
- **toString()** -- method returns a string with array values separated by commas.
- **join()** -- joins all array elements into a string. It behaves just like **toString()**, but you can specify the separator.
- **pop()** -- removes the last element from an array.
- **push()** -- adds a new element to an array (at the end).
- **shift()** -- removes the first element of an array, and "shifts" all other elements one place down.
- **unshift()** -- adds a new element to an array (at the beginning), and "unshifts" older elements.
- **splice()** -- method adds and/or removes array elements.
- **sort()** -- sorts an array alphabetically.
- **reverse()** -- reverses the elements in an array.
- **concat()** -- creates a new array by concatenating two arrays

See the complete list: http://www.w3schools.com/jsref/jsref_obj_array.asp

JAVASCRIPT OBJECTS

- An object is delimited by curly braces. Inside the braces the object's properties are defined as name and value pairs (name : value). The properties are separated by commas:
`var person={firstname:"John", lastname:"Doe", id:5566};`
 - Here, person is an Object with three properties
 - Firstname
 - Lastname
 - Id
 - And their values are
 - Firstname="John"
 - Lastname="Doe"
 - Id=5566
- Objects written as name value pairs are similar to:
 - Associative arrays in PHP
- In JavaScript, almost "everything" is an object.
 - Booleans can be objects (or primitive data treated as objects)
 - Numbers can be objects (or primitive data treated as objects)
 - Strings can be objects (or primitive data treated as objects)
 - Dates are always objects
 - Maths are always objects
 - Regular expressions are always objects
 - Arrays are always objects
 - Functions are always objects
 - Objects are objects

JAVASCRIPT OBJECT EXAMPLE

```
<!DOCTYPE html>
<html>
  <head> </head>
  <body>
    <script>
      var person = { firstname: "John", lastname: "Doe", id: 5566 };
      document.write(person.lastname + "<br>");
      document.write(person["firstname"] + " " + person.lastname);
    </script>
  </body>
</html>
```

Result / Output

Doe

John Doe

CONDITIONAL OPERATOR

- **Syntax**

variablename=(condition)?value1:value2

- **Example**

```
<script>  
  var age = 19;  
  voteable = age < 18 ? alert("Too young") : alert("Old enough");  
</script>
```

FUNCTIONS

```
<!DOCTYPE html>
<html>
  <body>
    <p>
      This example calls a function which performs a calculation, and returns
      the result:
    </p>
    <p id="demo"></p>
    <script>
      function myFunction(a, b) {
        return a * b;
      }
      document.getElementById("demo").innerHTML = myFunction(4, 3);
    </script>
  </body>
</html>
```

FUNCTIONS

```
<!DOCTYPE html>
<html>
<head>
  <script>
    function bodyload(){
      var x=document.getElementsByTagName("img");
      alert("total no.of images in page "+x.length);
    }
  </script>
</head>
<body onload="bodyload()">
  <div>
    
  </div>
</body>
</html>
```

FUNCTIONS

- In JavaScript, functions are objects.
- JavaScript functions have properties and methods.
- You can add your own properties and methods to functions.
- JavaScript Functions are Variables Too
- **In JavaScript, functions can be used as variables:**
 - Instead of

```
temp = toCelsius(32);  
text = "The temperature is " + temp + " Centigrade";
```
 - You can write

```
text = "The temperature is " + toCelsius(32) + " Centigrade";
```



```
function toCelsius(fahrenheit) {  
  return (5/9) * (fahrenheit-32);  
}
```

```
let value = toCelsius(77);
```

SCOPE

- In JavaScript, scope is the set of variables, objects, and functions you have access to.
 - Local : Variables declared within a JavaScript function, are LOCAL to the function
 - Global
 - Variables declared Globally (outside any block or function) have Global Scope.
 - Global variables can be accessed from anywhere in a JavaScript program.
 - Automatic Global
 - If you assign a value to a variable that has not been declared, it will become a GLOBAL variable.

This code example will declare a global variable **carName**, even if the value is assigned inside a function.

```
myFunction();  
  
// code here can use carName  
  
function myFunction() {  
  carName = "Volvo";  
}
```

Global Scope

```
<html>
  <head> </head>
  <body>
    <p id="demo"></p>
    <script>
      myFunction();
      document.getElementById("demo").innerHTML = "I can display " + carName;

function myFunction() {
  carName = "Volvo";
}
    </script>
    <!DOCTYPE html>
  </body>
</html>
```

carName is assigned without var, let, or const inside myFunction, it becomes an implicit global in non-strict mode and is therefore accessible after myFunction()

Local Scope

```
<script>
function myFunction() {
  var carName = "Volvo"; // local to myFunction
  document.getElementById("demo").innerHTML = "I can display " + carName;
}
myFunction();
</script>
```

JAVASCRIPT EVENTS

HTML events are "**things**" that happen to HTML elements.

When JavaScript is used in HTML pages, it can “react” to these events.

Event	Description
onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page

STRING METHODS

1. indexOf()
2. lastIndexOf()
3. search()

Extracting String Parts

- slice(start, end)
- substring(start, end)
- substr(start, length)

replace()

toUpperCase()

toLowerCase()

Extracting String Characters

- charAt(position)
- charCodeAt(position)

Converted to an array -- **split()**

```
<!DOCTYPE html>
<html>
<body>
<p id="demo"></p>
<script>
  var str = "Hello";
  var arr = str.split("");
  var text = "";
  var i;
  for (i = 0; i < arr.length; i++)
    { text += arr[i] + "<br>"
  }
  document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

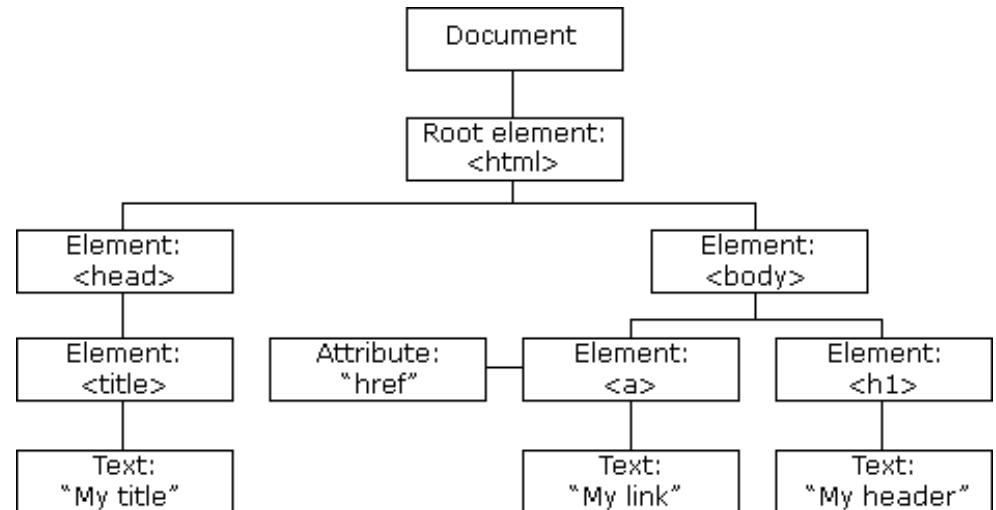
H
e
l
l
o

HTML DOM (DOCUMENT OBJECT MODEL)

DOM (DOCUMENT OBJECT MODEL)

When a web page is loaded, the browser creates a Document Object Model of the page.

The **HTML DOM** model is constructed as a **tree of Objects**:



* **With a programmable object model, JavaScript gets all the power it needs to create dynamic HTML:**

- * JavaScript can change all the **HTML elements** in the page
- * JavaScript can change all the **HTML attributes** in the page
- * JavaScript can change all the **CSS styles** in the page
- * JavaScript can **react to all the events** in the page

FINDING HTML ELEMENTS

- There are few ways to find HTML elements
 - **Finding HTML elements by id**
 - `var x=document.getElementById("intro");`
 - Here, intro is an “id” of a html element.
 - Example: `<p id=“intro”>Hello World</p>`
 - **Finding HTML elements by tag name**
 - `var y=x.getElementsByTagName("p");`
 - Here, p is html tag `<p> </p>`
 - `getElementsByTagName` returns array
 - Commonly used for select all checkbox
 - **`document.getElementById('').Value`** for input,textarea
 - **`document.getElementById('').innerHTML`** for div, span, th, td

Changing the Value of an Attribute

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```

```

```
<script>
```

```
document.getElementById("myImage").src = "landscape.jpg";
```

```
</script>
```

```
</body>
```

```
</html>
```

InnerHTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>innerHTML Example</title>
</head>
<body>
  <h2 id="greeting">Hello Students!</h2>
  <button onclick="changeText()">Click Me</button>

  <script>
    function changeText() {
      document.getElementById("greeting").innerHTML = "Welcome to JavaScript Class!";
    }
  </script>
</body>
</html>
```


Value

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>getElementById - Value Example</title>
  </head>
  <body>
    <h2>Enter Your Name:</h2>

    <input type="text" id="username" placeholder="Type your name here" />
    <button onclick="showName()">Submit</button>

    <h3 id="display"></h3>

    <script>
      function showName() {
        let name = document.getElementById("username").value;
        document.getElementById("display").innerHTML = "Hello, " + name + "!";
      }
    </script>
  </body>
</html>
```

HTML DOM EVENTS

A JavaScript can be executed when an event occurs, like

- When a user clicks on an HTML element = onclick
- When a web page has loaded = onload
- When the mouse moves over an element = onMouseMove
- When an input field is changed=onchange
- When a user strokes a key = onKeyPress, onKeyUp
- When the user double-clicks = ondblclick

For all events, declare

- eventname=*JavaScript function*

UNDERSTANDING FORM VALIDATION

- A form validation is a process by which the server or the browser checks form entries and, where possible, eliminates errors.
- On the Web, validation can occur on the client or server side. Form validation is a critical aspect of data entry.
- A properly designed form reduces the possibility of faulty data being entered.
- A powerful use of JavaScript is to provide client-side validation.
- Using a script built into the Web page form provides immediate feedback to users as they enter data.
- Client-side validation can reduce the network traffic between users and the Web server.

SERVER-SIDE AND CLIENT-SIDE VALIDATION



Server-side validation

- 1) The user submits the form to the Web server.
- 2) The Web server validates the user's responses and, if necessary, returns the form to the user for correction.
- 3) After correcting any errors, the user resubmits the form to the Web server for another validation.



Client-side validation

- 1) The user submits the form, and validation is performed on the user's computer.
- 2) After correcting any errors, the user submits the form to the Web server.

SUBMITTING & CONTROLLING A FORM

- If a condition of the form is not met, the browser should refuse the form submission and indicate to the user why the form was not submitted.
- When a user completes a form and then clicks the submit button, a submit event is initiated.
- JavaScript provides the onsubmit event handler that allows you to run a program in response to this action.
- The submit event is associated with the form object, the event handler must be placed within the <form> tag.
- The onsubmit event handler must be able to override the act of submitting the form if the form fails a validation test.

|

Thank You